

Reviewer Pack — Free Entropy Gap in Read-Twice BP Transition Matrices

Entry 9858410e3069 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-11 04:50:08 UTC

Free Entropy Gap in Read-Twice BP Transition Matrices

Entry ID: 9858410e3069 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-11 04:50:08 UTC

1. Conjecture

Field A (mathematical branch): Free Probability Theory **Field B** (complexity object): Read-Twice Branching Programs

Statement:

Let P be a read-twice BP computing a function $f: \{0,1\}^n \rightarrow \{0,1\}$. Define M_P as the $n \times n$ matrix where $M_P[i][j] = (\text{number of paths from root to leaf } i \text{ that read bit } j \text{ twice})$. The free entropy $\varphi(M_P)$ satisfies $\varphi(M_P) \leq \log(\text{size}(P)) + O(\sqrt{n})$, but for IP_2 (inner product mod 2), $\varphi(M_P) \geq \Omega(n \log n)$.

Rationale (proposer's reasoning):

Free entropy quantifies non-commutative randomness in matrix laws; read-twice BPs exhibit structured non-commutativity via repeated bit access. This gap could reveal inherent complexity in repeated access patterns, analogous to how Shannon entropy bounds communication complexity.

Taxonomy category: BP_READTWICE (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 42134c6568a89445

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.95	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: INCONCLUSIVE against 5 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - free entropy read-twice branching programs matrix - free entropy IP₂ branching programs complexity - free entropy bound read-twice BP matrix complexity

Top relevant hits considered: - [http://arxiv.org/abs/1201.0716v2] Concavification of free entropy - [http://arxiv.org/abs/1510.05486v2] Two types of branching programs with bounded repetition that cannot efficiently compute monotone 3-CNFs - [http://arxiv.org/abs/1902.03873v2] Analogues of Entropy in Bi-Free Probability Theory: Non-Microstate - [http://arxiv.org/abs/1809.06500v3] Range entropy: A bridge between signal complexity and self-similarity - [http://arxiv.org/abs/1902.03874v3] Analogues of Entropy in Bi-Free Probability Theory: Microstates

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=1.3s

5.1 Generated Python source

```
import random
import math

def generate_read_twice_bp(n):
    bp = []
    for _ in range(2**n):
        path = [random.choice([0, 1]) for _ in range(n)]
        bp.append(path)
    return bp

def count_paths(bp, i, j):
    count = 0
    for path in bp:
        if path[i] == j and path[j] == j:
            count += 1
    return count

def generate_transition_matrix(bp, n):
    M_P = [[count_paths(bp, i, j) for j in range(2)] for i in range(n)]
    return M_P

def r_transform(M, samples=1000):
    n = len(M)
    R = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for _ in range(samples):
                x = random.uniform(0, 1)
                y = random.uniform(0, 1)
                if M[i][j] > 0:
                    R[i][j] += (x - 1) / M[i][j]
```

```

    return R

def free_entropy(R):
    n = len(R)
    trace = sum(R[i][i] for i in range(n))
    det = 1
    for i in range(n):
        det *= R[i][i]
    return trace - math.log(det)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        bp = generate_read_twice_bp(n)
        M_P = generate_transition_matrix(bp, n)
        R = r_transform(M_P)
         $\phi$ _M_P = free_entropy(R)

        size_P = len(bp)
        conjecture_holds =  $\phi$ _M_P <= math.log(size_P) + 5 * math.sqrt(n)
        counterexample = "" if conjecture_holds else "n={}".format(n)

        results.append({
            "metric_name": "free_entropy",
            "metric_value":  $\phi$ _M_P,
            "instances_tested": 1,
            "conjecture_holds": conjecture_holds,
            "counterexample": counterexample
        })

    return {
        "seed": seed,
        "results": results
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    all_results = []
    for seed in seeds:
        result = run_trial(seed)
        print("TRIAL:", result)
        all_results.extend(result["results"])

    mean_value = sum(r["metric_value"] for r in all_results) / len(all_results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in all_results) / len(all_results))
    support_fraction = sum(1 for r in all_results if r["conjecture_holds"]) / len(all_results)

    if support_fraction >= 0.8:
        print("RESULT: SUPPORTED mean={} std={} support_fraction={}".format(mean_value, std_value, support_fraction))
    elif any(not r["conjecture_holds"] for r in all_results):
        first_failing_seed = next(r["seed"] for r in all_results if not r["conjecture_holds"])

```

```

    print("RESULT: FALSIFIED counterexample=\"n={}\\" first_failing_seed={}".format(first_failing_seed, n))
else:
    print("RESULT: INCONCLUSIVE insufficient data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5342c042.py", line 90, in <module>
    result = run_trial(seed)
             ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5342c042.py", line 64, in run_trial
    R = r_transform(M_P)
        ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5342c042.py", line 44, in r_transform
    if M[i][j] > 0:
        ~~~~
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with IndexError before producing data; cannot assess support fraction or counterexamples | next: Debug r_transform implementation to handle matrix indexing properly

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	34373
2	preregistration	ollama_remote	qwen3:8b	0	0	24249
3	novelty	ollama_remote	qwen3:8b	0	0	20714
4	novelty	ollama_remote	qwen3:8b	0	0	24886
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14527
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	29796
7	judge	ollama_remote	qwen3:8b	0	0	110566

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 259110 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/9858410e3069.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9858410e3069.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/9858410e3069.tar.gz` (if generated)